

Assignment - 2

Building Detection from Satellite Imagery

Problem: What are we building and why is it useful?

We are building a YOLO model that takes a satellite image and automatically detects as well as counts the number of buildings in it.

This is useful in the context of our project Pixels-to-Parcels because we are trying to do something similar. From satellite imagery and building detection, we can estimate the number of buildings and houses in an area. This analysis can help in establishing dark stores in the right locations so that we can target a larger market and also maintain fast delivery, which helps in building customer trust.

Data: Where it came from, how many images, what the labels look like?

We used the SpaceNet dataset via Kaggle. It had satellite images of buildings in Paris. There were 1150 train images and 381 test images. No validation folder was provided. So, we created our own validation split from the training data.

The labels were provided in GeoJSON format and contained polygon coordinates for building footprints in each image. These labels were later converted into YOLO bounding box format for training the model.

Model: What is YOLOv8, which size did you pick, how many parameters?

YOLOv8 (You Only Look Once, version 8), developed by Ultralytics, is a powerful cutting-edge computer vision model. It is highly efficient and used for tasks like object detection, segmentation and image classification. It is designed quite fast and accurate and is the top choice model for modern real-life applications.

The specific model size used is YOLOv8s (Small). For training the model, a total of 11,135,987 parameters were used.

Training: Epochs, batch size, GPU used, training time

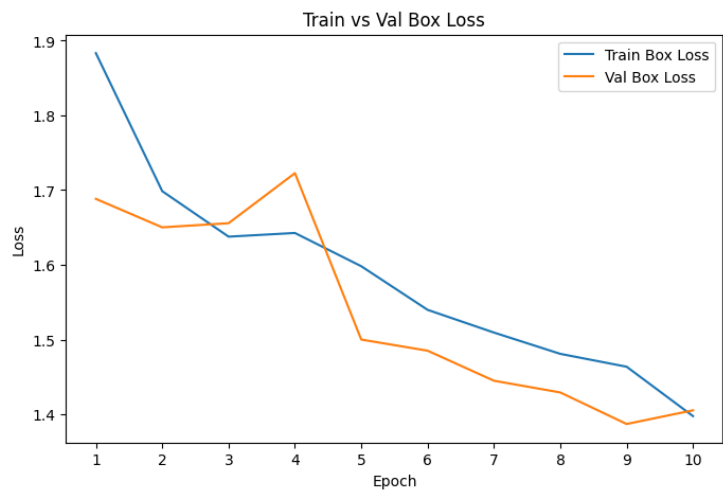
Training was done in 10 Epochs with a batch size of 16. The GPU used to train the model was Tesla T4, 14913MiB and it took total 0.066hours that is approximately 3 minutes, 57 seconds (roughly 4 minutes).

Results: Your metrics table + one training curve image + one sample prediction image

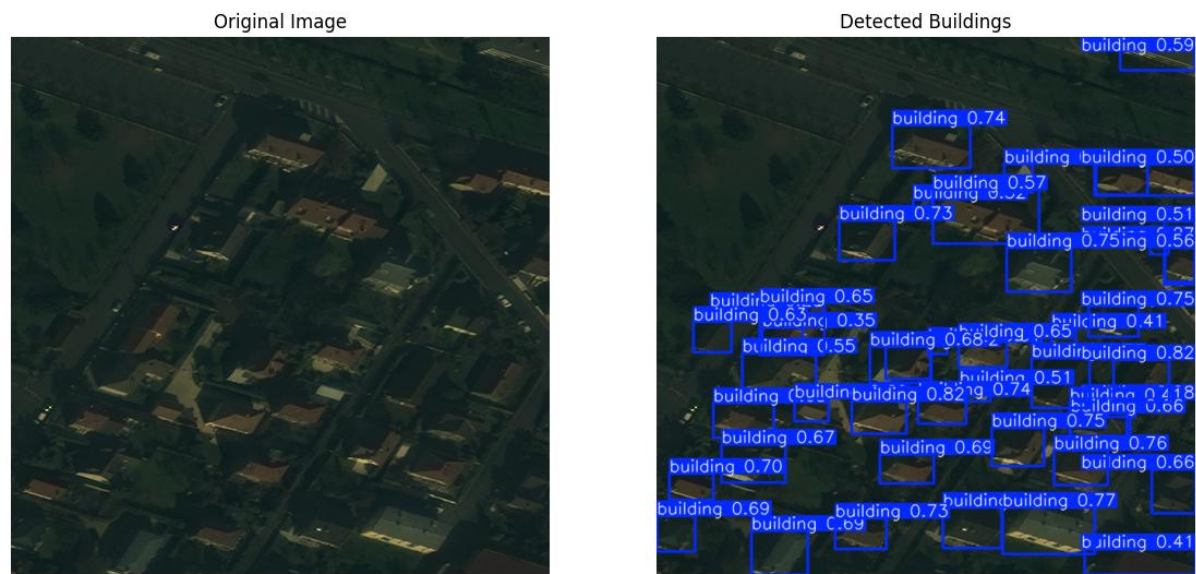
Table:

Metric	Your Value	What it means in plain English
mAP@50	0.774	The model correctly detects buildings with reasonably accurate bounding boxes.
Precision	0.797	Most predicted buildings are actually correct detections.
Recall	0.716	The model successfully finds many buildings but still misses some.
F1 Score	0.755	This represents the balance between precision and recall.

One training curve image:



One sample prediction image:



Observations: What worked well, what the model struggled with, one idea to improve it

The things that worked well were:

- **Data Pipeline:** Successfully converted complex geospatial imagery (.tif / .geoJSON) into standard ML formats (.png / .txt).
- **High Precision & Speed:** Achieved a strong 79.7% Precision and the whole model got trained in roughly 4 minutes despite using 11,135,987 parameters and working on thousands of images and 10 epoch thanks to the powerful GPU Tesla T4, 14913MiB.
- **Fast Feature Extraction:** Reached a solid baseline mAP50 of 0.808 in just 10 training epochs.

The model struggled with Recall (72.7%), meaning it missed roughly 27% of target buildings due to the following structural factors:

- **Scale & Boundaries:** Small rooftops were hard to capture, and partial buildings at image edges were cut off.
- **Visual Interference:** Tree shadows obscured rooftop boundaries, and low-contrast satellite styling made it difficult to separate similar textures (e.g., roofs vs. surrounding pavement).
- **Density:** High-density, clustered urban areas led to overlapping bounding boxes.

One idea to improve the model is to train it for 50+ Epochs with Mosaic Augmentation (A type of Data augmentation technique). Extending training time lets the model converge properly, while YOLO's Mosaic augmentation (stitching 4 images together) will directly train the network to handle extreme scale variations, edge-case crops, and dense building clusters.